Pablo Mendez - 23975

Hoja de trabajo 2

Parte 1:

```
paulm@paulm:~$ mpirun --oversubscribe -np 4 ./parte1_openmpi
Rank 0 comunicador
Rank 1 con size 4
Rank 2 con size 4
Rank 3 con size 4
```

¿Qué significa rank y size en el contexto de este programa?

rank: es el identificador único de cada proceso dentro del comunicador MPI_COMM_WORLD. Va desde 0 hasta size - 1.

size: es la cantidad total de procesos que están ejecutando el programa dentro del comunicador. En el programa, N corresponde al size, es decir, al número de procesos lanzados con mpiexec -np N.

¿Qué pasa si ejecutas el programa con 1 núcleo?

Se ejecuta con size = 1 y solo existe el rank 0. La salida sería:

Rank 0 comunicador

No se imprimen más ranks porque el ciclo va desde 1 hasta size - 1, y si size = 1, el ciclo no se ejecuta. No hay paralelismo real, pero MPI funciona correctamente con un solo proceso.

Parte 2:

```
paulm@paulm:~$ mpiexec --oversubscribe -np 4 ./parte2
Etapa 1: Rank 2 de 4 antes de MPI_Barrier
Etapa 1: Rank 3 de 4 antes de MPI_Barrier
Etapa 1: Rank 0 de 4 antes de MPI_Barrier
Etapa 1: Rank 1 de 4 antes de MPI_Barrier
Etapa 2: Rank 3 de 4 después de MPI_Barrier
Etapa 2: Rank 0 de 4 después de MPI_Barrier
Etapa 2: Rank 2 de 4 después de MPI_Barrier
Etapa 2: Rank 1 de 4 después de MPI_Barrier
```

¿Qué sucede antes y después de la barrera?

Antes de MPI_Barrier: cada proceso avanza de forma independiente. Algunos pueden llegar más rápido que otros.

Después de MPI_Barrier: ningún proceso puede continuar hasta que todos los procesos del comunicador hayan llegado a la barrera. Cuando el último proceso llega, todos se liberan y

continúan.Por eso, la Etapa 1 queda completamente terminada antes de que cualquier proceso empiece la Etapa 2.

¿Para qué sirve MPI_Barrier?

Sirve para sincronizar todos los procesos de un comunicador. Es un punto de espera colectivo. No envía datos, solamente detiene a los procesos hasta que todos hayan alcanzado la barrera. Se usa cuando se necesita que una fase termine antes de iniciar la siguiente.

Parte 3:

```
paulm@paulm:~$ mpiexec --oversubscribe -np 4 ./parte3
Rank 2 recibió valor = 99999
Rank 0 (coordinador) asigna valor = 99999
Rank 0 recibió valor = 99999
Rank 1 recibió valor = 99999
Rank 3 recibió valor = 99999
```

En tus palabras, ¿qué hace cada instrucción?

MPI_Bcast: es una operación colectiva de difusión. Un proceso raíz, en este caso el rank 0, envía el mismo dato a todos los demás procesos del comunicador. Todos los procesos deben llamar a MPI_Bcast con los mismos argumentos.

MPI_INT: es el tipo de dato de MPI que corresponde a un int de C. Le indica a MPI que el buffer contiene enteros.

Cambie el valor inicial por uno muy grande, por ejemplo 99999. ¿Qué sucede?

Con 99999 no hay problema porque cabe en un int normal de 32 bits. El valor máximo de un int con signo suele ser 2147483647. Si se usa un número mayor que ese, habría desbordamiento. Si se necesitan valores más grandes, se debe usar long long y el tipo MPI_LONG_LONG.

```
paulm@paulm:~$ mpiexec --oversubscribe -np 4 ./parte3
Rank 2 recibió valor = 9999999999
Rank 3 recibió valor = 9999999999
Rank 0 (coordinador) asigna valor = 9999999999
Rank 0 recibió valor = 9999999999
Rank 1 recibió valor = 9999999999
```

Al usar un valor mayor al rango de int, que en esta plataforma es 2,147,483,647, es necesario cambiar el tipo de dato a long long y el tipo MPI a MPI_LONG_LONG. Si solo se cambia el valor sin cambiar el tipo, se produce un desbordamiento, un overflow, y los procesos reciben un número incorrecto. Con long long, el valor 9999999999 se difunde correctamente a todos los procesos.

Pablo Mendez - 23975

Diagrama de flujo de cómo MPI_Bcast ayuda a la asignación de un valor a cada proceso

